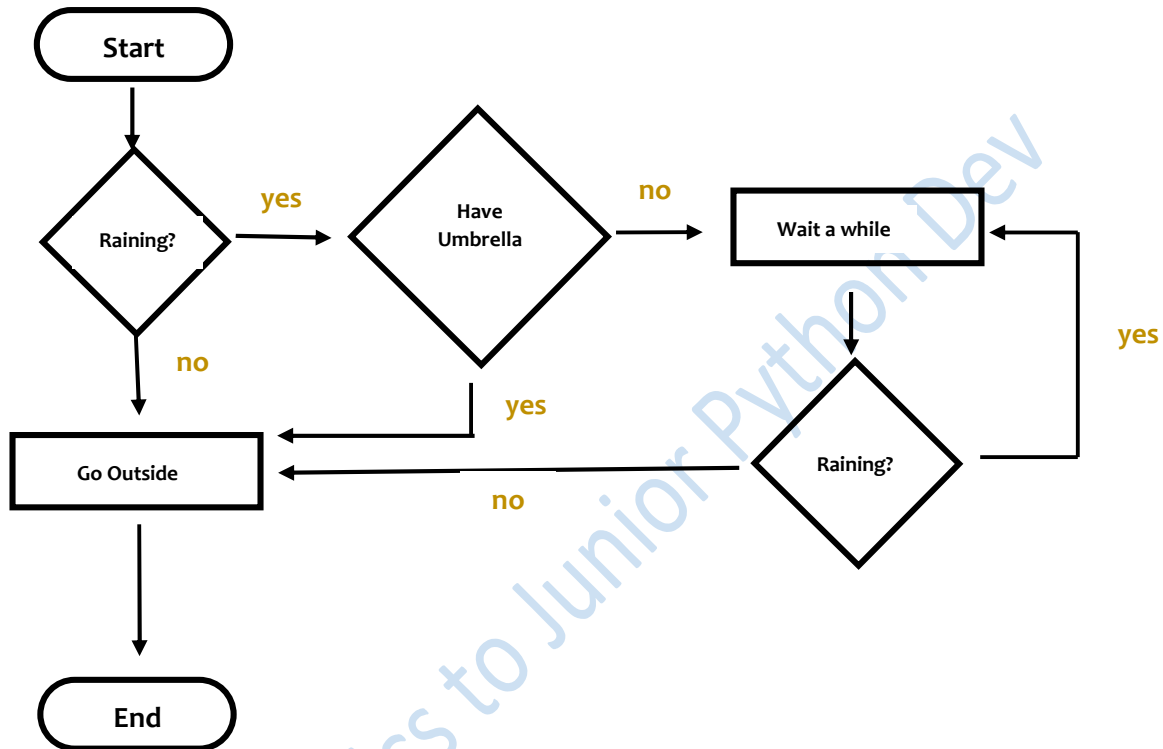


FROM BASICS TO JUNIOR PYTHON DEV: Conditionals (A)

Programming often involves looking at a set of conditions and responding appropriately based on how these conditions evaluate. The program decides to execute some instructions, repeat them, skip them or choose one of the given conditions. Conditionals are used in such cases and decide which instructions the program should skip or run.

Consider the flow chart below:



In this flow chart, there's more than one way to reach the end. This is the same case when it comes to programming. You would like some instructions to be executed only when a specific condition is satisfied.

Before we dive into conditionals, you first need to learn how to represent the **yes** or **no** as well as understand how to write those **branching points** in python code. Therefore, we'll have to look at:

- **Boolean Values**
- **Comparison Operators**
- **Boolean Operators**

Boolean Values

As said in our previous lesson, a Boolean data type has only two values; **True** & **False**. When typing these values as Python code, quotes are excluded and they start with an uppercase letter, T or F.



FROM BASICS TO JUNIOR PYTHON DEV: Conditionals (A)

Note that these values cannot be used as variable names. Python will give you an error if it encounters such an assignment.

```
C:\Users\njrus\Desktop\python work\booleans.py - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
booleans.py
1
2 # Boolean values cannot be used as variable names
3
4 True = '1+2'
5
6 print(True)

File "C:\Users\njrus\Desktop\python work\booleans.py", line 4
True = '1+2'
^^^^
SyntaxError: cannot assign to True
[Finished in 382ms]
```

Comparison Operators

Comparison Operators are used to compare two values and evaluate to a single Boolean value. That is, they compare two values against some condition and evaluate to **True** if the condition is satisfied else **False**.

The table below shows the comparison operators.

Symbol	Meaning
==	Equal to
!=	Does not equal to
<	Less Than
>	Greater than
<=	Less or equal to
>=	Greater or equal to

We can try using them in an interactive shell.

```
Python 3.12 (64-bit)
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> 12 == 12
True
>>> 13 <= 12
False
>>> 'python' == 'Python'
False
>>> 49 <= 49
True
>>> (7 - 3) == (10 - 6)
True
>>> 10 < 7
False
>>>
```



You'll often encounter these operators in conditional statements so it is important you familiarize yourself with them.

Boolean Operators

Python has three Boolean operators; **and**, **or** & **not**. They are used to compare Boolean values (True and False). Just like comparison operators, they reduce expressions to a single Boolean value.

The and & or Operators

The **and** & **or** operators always take two Boolean values/expressions.

The table below summarizes the **and** operator.

Expression	Evaluates to
True and True	True
True and False	False
False and False	False
False and True	False

From this table, we can conclude that the **and** operator evaluates an expression to **True** if both Boolean values are True, else it evaluates to False.

Let us now have a look at the **or** operator.

Expression	Evaluates to
True or True	True
True or False	True
False or False	False
False or True	True

The **or** operator evaluates an expression to True if one of the Boolean values is True. Only when both expressions are False that it evaluates to False.

The not Operator

Unlike the **and** & **or** operators, the **not** operator operates on one Boolean value/expression. The **not** operator usually evaluates to the opposite Boolean value.

The table below summarizes the **not** operator.



Expression	Evaluates to
not True	False
not False	True

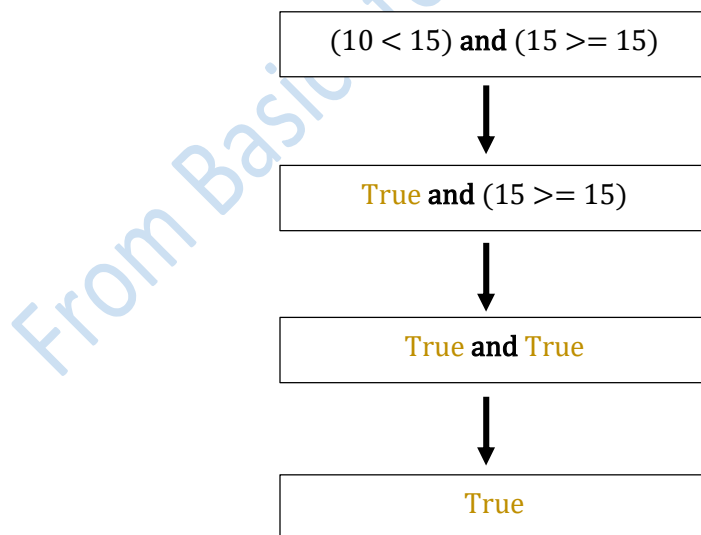
Mixing Boolean and Comparison Operators

Since comparison operators evaluate to Boolean values, you can use them in expressions with the Boolean operators. Recall that the **and**, **or**, and **not** operators are called Boolean operators because they always operate on the Boolean values **True** and **False**. While expressions like `4 < 5` aren't Boolean values, they are expressions that evaluate down to Boolean values.

We can try entering a Boolean expression that use comparison operators into the interactive shell.

```
Python 3.12 (64-bit)
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> (10 < 15) and (15 >= 15)
True
>>> _
```

Python will evaluate the left expression first, and then it will evaluate the right expression. When it knows the Boolean value for each, it will evaluate the whole expression down to one Boolean value. Conceptually, you can think of this process as;



Finally, Boolean operators have an order of operations just like the math operators do. Python evaluates the **not** operators first, then the **and** operators, and then the **or** operators.



Elements of Flow Control

Flow Conditional Statements (Conditionals) often start with a part called the **condition**, and all are followed by a block of code called the **clause**. Let's have a look at what a condition & clause are;

Condition

The Boolean expressions you've encountered so far could all be considered as conditions, which are the same thing as expressions; a **condition** is just a more specific name in the context of flow control statements.

Conditions always evaluate down to a Boolean value, **True** or **False**. A flow control statement decides what to do based on whether its condition is True or False, and almost every flow control statement uses a condition.

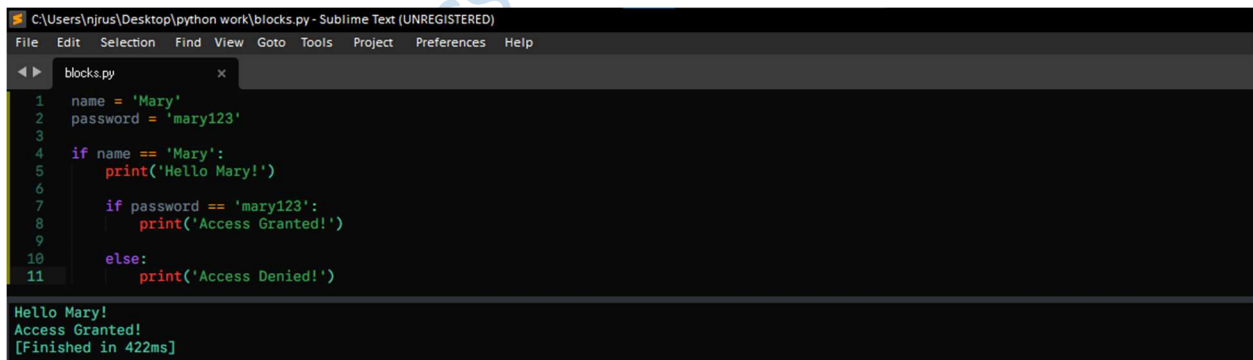
Blocks of Code/Clause

Lines of Python code can be grouped together in clauses. Python executes instructions in a clause if the underlying condition is True.

There are three rules for clauses;

1. Clauses begin when the indentation increases.
2. Clauses can contain other blocks.
3. Clauses end when the indentation decreases to zero or to a containing block's indentation.

You can tell when a clause begins and ends from the indentation of the lines of code:



```
C:\Users\njrus\Desktop\python work\blocks.py - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
blocks.py
1 name = 'Mary'
2 password = 'mary123'
3
4 if name == 'Mary':
5     print('Hello Mary!')
6
7     if password == 'mary123':
8         print('Access Granted!')
9
10    else:
11        print('Access Denied!')
```

Hello Mary!
Access Granted!
[Finished in 422ms]

The first clause of code at 5 starts at the line **print('Hello Mary')** and contains all the lines after it.

Inside this clause is another clause at 8, which has only a single line in it: **print('Access Granted.')**. The third clause at 11 is also one line long: **print('Wrong password.')**

